

ULB StudyShare

Architecture de l'Application

Python + Tkinter

MySQL

INFO-H303

Présentation Générale

ULB StudyShare est une application de partage de résumés de cours entre étudiants ULB. Elle permet de publier, consulter, évaluer des résumés, gérer un profil avec points et cosmétiques, et visualiser des statistiques avancées.



Authentification

Inscription, connexion, déconnexion

Tableau de bord

Profil, points, niveau, activités récentes



Explorateur

Recherche de cours et résumés



Ma Bibliothèque

Consultation, édition, suppression de ses résumés



Boutique

Badges, titres et thèmes cosmétiques



Statistiques

8 requêtes SQL demandées par le projet

Architecture Logicielle

Organisation en couches — MVC adapté

Architecture en Couches

GUI — Interface Tkinter

Vues auth / workspace · Contrôleurs · Transitions

1

Services — Logique Métier

Règles métier · Validation · Gestion des points

2

Repository — Accès Données

Requêtes SQL · CRUD par domaine · 6 fichiers réels

3

Base de Données — MySQL

schema.sql · Triggers · Contraintes · 7 entités

4

Flux d'exécution : GUI → Services → Repository → MySQL

Couche GUI — Interface Tkinter

Structure gui/

app.py	Création fenêtre Tkinter principale
controller.py	Contrôleur principal — état global
controllers/	AuthController · WorkspaceController
views/auth/	Écrans login & inscription
views/workspace/	Dashboard · Explorer · Boutique...
views/common/	Thème · Sidebar partagée
ui_helpers.py	Chargement assets & icônes
transitions.py	Animations entre vues

Principes clés

Séparation des responsabilités

Les vues ne contiennent aucun SQL. Elles reçoivent des DTOs du contrôleur et affichent sans décider.

Contrôleur unique

Conserve l'utilisateur connecté et orchestre le rafraîchissement des vues après chaque action.

Navigation centralisée

Transitions gérées par transitions.py — aucune logique de routage dans les vues.

Thème global

Palette de couleurs et polices définies dans views/common/.

Services & Repositories

core/services/

Logique Métier

- ✓ Vérification utilisateur connecté
- ✓ Champs obligatoires validés
- ✓ Interdiction d'auto-évaluation
- ✓ Points suffisants pour achat
- ✓ Objet possédé avant activation
- ✓ Attribution des récompenses en points
- ✓ Calcul du niveau & titre actif

core/repository/

Accès aux Données SQL

- explorer_repository.py** Recherche cours, résumés
- my_library_repository.py** CRUD résumés de l'utilisateur
- user_repository.py** Profil, auth, points, niveau
- shop_repository.py** Objets cosmétiques, possessions
- leaderboard_repository.py** Classement & points
- statistics_repository.py** Requêtes statistiques du projet

Base de Données MySQL

Schéma, contraintes et triggers

Entités de la Base de Données

Utilisateurs

id · pseudo · email · mot de passe
points · niveau · titre_actif

Cours

code · nom · faculté · crédits
année académique

Résumés

id · titre · contenu
auteur_id · cours_id · année

Évaluations

id · note (1–5) · commentaire
auteur_id · résumé_id

Objets cosmétiques

id · type (badge/titre/thème)
nom · coût en points

Possessions

utilisateur_id · objet_id
actif · date achat

Transactions points

id · utilisateur_id · montant
raison · horodatage

Contraintes & Triggers SQL

PRIMARY KEY

`utilisateur.id, résumé.id, évaluation.id, ...`

FOREIGN KEY

`résumé.auteur_id → utilisateur.id, ...`

CHECK

`note BETWEEN 1 AND 5, crédits > 0, ...`

UNIQUE

`email unique, (auteur_id, résumé_id) unique, ...`

NOT NULL

`pseudo, email, mot de passe, titre résumé, ...`

Triggers SQL

Attribution automatique de points • Mise à jour du niveau utilisateur • Vérification d'intégrité avant insertion

Import des Données

CSV · XML · JSON → MySQL

Pipeline d'Import de Données

CSV	XML	XML	JSON
cours.csv	recompenses.xml	utilisateurs/	commentaires.json
47 / 47	45 / 45	50 / 50	87 / 94
cours importés	objets cosmétiques	utilisateurs + résumés	évaluations (87/94)

▼ Pipeline d'import



Lancé avec : `python3 main.py --init`

Choix Techniques

Justifications des technologies utilisées

Justifications des Choix Techniques

Python 3.10+

Libre choix technologique. Python s'est imposé naturellement pour sa lisibilité, son excellent support de MySQL (mysql-connector-python) et son écosystème riche.

Tkinter

Interface native incluse avec Python — aucune dépendance externe. Simple, portable, suffisant pour une application desktop académique.

MySQL (vs PostgreSQL)

PostgreSQL était suggéré par l'énoncé, mais nous avons choisi MySQL : il supporte pleinement les triggers, CHECK et UNIQUE requis, et notre équipe le maîtrisait mieux.

Architecture en couches

Séparation claire GUI / Services / Repository. Facilite la maintenance, évite le SQL dans les vues, et respecte le principe de responsabilité unique.

Fichiers .env

Connexion DB externalisée via python-dotenv. Aucun secret dans le code source, configuration simple pour chaque membre de l'équipe.

Requêtes SQL

Les 8 requêtes demandées par l'énoncé INFO-H303

Vue d'ensemble — 8 Requêtes SQL

1	Top 10 utilisateurs par points	Simple	2	Publiés dans ≥ 3 cours différents	Moyenne
3	Cours avec le plus de résumés	Simple	4	Résumés les mieux notés par cours	Avancée
5	Utilisateurs sans résumé publié	Simple	6	Objet cosmétique le plus acheté	Simple
7	Dépensé plus que solde disponible	Avancée	8	Moyenne de résumés par utilisateur	Moyenne

Simple

Moyenne

Avancée

— Les 4 requêtes encadrées ci-dessous sont détaillées dans les slides suivantes

SQL

```
SELECT rr.codeCours, rr.nomCours, rr.titre, rr.avg_note
FROM (
    SELECT c.codeCours, c.nomCours, r.idResume, r.titre,
           AVG(e.note) AS avg_note
    FROM Resume r
    JOIN Cours c ON c.codeCours = r.codeCours
    JOIN Evaluer e ON e.idResume = r.idResume
    GROUP BY c.codeCours, c.nomCours, r.idResume, r.titre
) AS rr
JOIN (
    SELECT codeCours, MAX(avg_note) AS best_avg
    FROM ( SELECT c.codeCours, AVG(e.note) AS avg_note
          FROM Resume r JOIN Cours c ... JOIN Evaluer e ...
          GROUP BY c.codeCours, r.idResume ) AS avg
    GROUP BY codeCours ) AS best_per_course
```

```
ON best_per_course.codeCours = rr.codeCours
```

Sous-requête 1 (bleue)

Calcule la note moyenne $AVG(e.note)$ de chaque résumé en joignant Resume, Cours et Evaluer.

Sous-requête 2 (jaune)

Re-calcule les moyennes par résumé, puis prend le MAX par cours — le meilleur score de chaque cours.

Jointure finale (verte)

Filtre : garde uniquement les résumés dont la moyenne = best_avg pour ce cours. Plusieurs ex-aequo possibles.

Complexité

3 niveaux d'imbrication. Impossible à exprimer avec un seul GROUP BY — nécessite deux passes sur les données.

Utilisateurs ayant publié dans ≥ 3 cours différents

GROUP BY + HAVING + COUNT(DISTINCT) — agrégation avec filtre sur agrégat

SQL

```
SELECT u.nomUtilisateur,  
       COUNT(DISTINCT r.codeCours) AS nb_cours,  
       COUNT(r.idResume)          AS nb_resumes  
FROM Utilisateur u  
JOIN Resume r ON r.idUtilisateur = u.idUtilisateur  
  
GROUP BY u.idUtilisateur, u.nomUtilisateur  
  
HAVING COUNT(DISTINCT r.codeCours) >= 3  
  
ORDER BY nb_cours DESC, nb_resumes DESC,  
         u.nomUtilisateur ASC
```

COUNT(DISTINCT codeCours)

Ne compte chaque cours qu'une seule fois, même si l'utilisateur y a publié plusieurs résumés.

JOIN Resume (jaune)

Seuls les utilisateurs ayant au moins un résumé entrent dans l'agrégat. Ceux sans résumé sont exclus.

GROUP BY (vert)

Regroupe par utilisateur pour calculer les compteurs. On inclut idUtilisateur pour éviter les doublons de nom.

HAVING (rouge)

Filtre APRES agrégation — WHERE ne peut pas filtrer sur COUNT. HAVING s'applique aux groupes formés.

Utilisateurs ayant dépensé plus que leur solde

LEFT JOIN + COALESCE + HAVING sur agrégat — filtrage post-agrégation cross-table

SQL

```
SELECT u.nomUtilisateur, u.nombrePoints,  
       COALESCE(SUM(tp.montantPoints), 0) AS total_spent,  
       COALESCE(SUM(tp.montantPoints), 0)  
         - u.nombrePoints                AS excess_spent  
FROM Utilisateur u  
LEFT JOIN TransactionPoints tp  
  ON tp.idUtilisateur = u.idUtilisateur  
  AND tp.natureTransaction = 'depense'  
  
GROUP BY u.idUtilisateur, u.nomUtilisateur, u.nombrePoints  
  
HAVING COALESCE(SUM(tp.montantPoints), 0) > u.nombrePoints  
ORDER BY excess_spent DESC, total_spent DESC
```

COALESCE(SUM(...), 0)

Si l'utilisateur n'a aucune dépense (LEFT JOIN = NULL), SUM retourne NULL. COALESCE le ramène à 0 pour la comparaison.

LEFT JOIN + AND (jaune)

Le filtre natureTransaction='depense' est dans le ON, pas le WHERE — sinon les utilisateurs sans dépense seraient exclus.

GROUP BY (vert)

Agrège toutes les transactions de dépense par utilisateur pour obtenir le total dépensé.

HAVING (rouge)

Compare le total dépensé au solde actuel. Un résultat > 0 indique un utilisateur en déficit de points.

Nombre moyen de résumés par utilisateur

Sous-requête dans le FROM (table dérivée) — AVG sur résultat d'un GROUP BY

SQL

```
SELECT AVG(user_resume_count)
FROM (

    SELECT u.idUtilisateur,
           COUNT(r.idResume) AS user_resume_count
    FROM Utilisateur u
    LEFT JOIN Resume r
        ON r.idUtilisateur = u.idUtilisateur
    GROUP BY u.idUtilisateur

) AS per_user
```

Pourquoi une sous-requête ?

On ne peut pas écrire `AVG(COUNT(...))`. Il faut d'abord calculer les compteurs par utilisateur, puis en faire la moyenne.

LEFT JOIN (jaune)

Inclut les utilisateurs sans aucun résumé (`COUNT = 0`). Sans `LEFT JOIN`, ils seraient exclus et la moyenne serait biaisée vers le haut.

Table dérivée AS per_user (verte)

La sous-requête produit une table temporaire avec une ligne par utilisateur et son nombre de résumés.

Résultat

Retourne un seul float : le nombre moyen de résumés publiés en comptant TOUS les utilisateurs, y compris ceux à 0.

Utilisation de l'IA

Assistance ponctuelle sur des sujets hors programme

Utilisation de l'IA dans le Projet

L'IA a été utilisée de façon ponctuelle, uniquement sur des sujets peu ou pas couverts en cours. Toutes les décisions d'architecture, de modélisation et de requêtes SQL restent notre travail.

Index MySQL

Les index ne sont pas couverts en cours (INFO-H303). Nous avons utilisé l'IA pour comprendre quand et comment créer des index pertinents sur nos tables (ex. sur `auteur_id`, `cours_id`) afin d'optimiser les requêtes fréquentes.

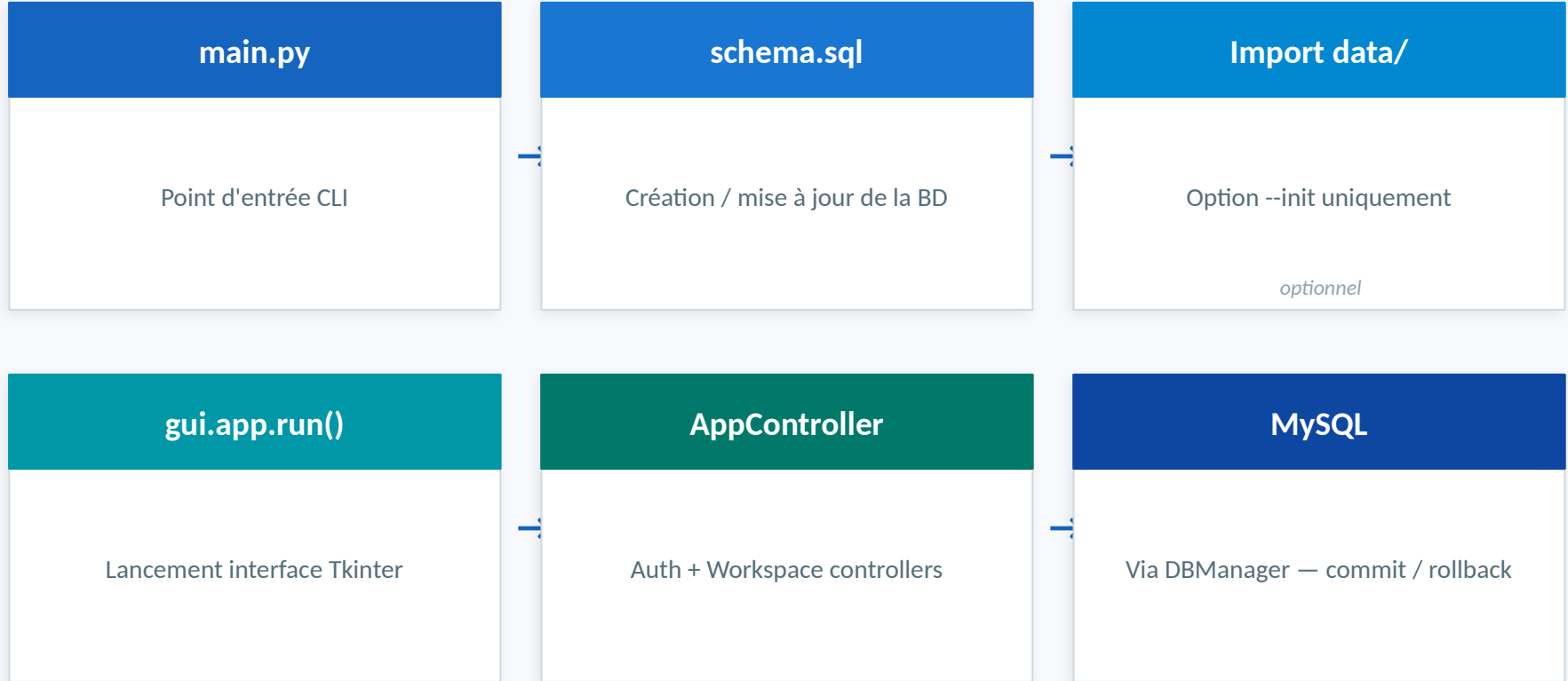
Triggers SQL

Les triggers sont abordés en cours mais pas pratiqués en TP. Nous avons utilisé l'IA pour nous aider à rédiger les triggers d'attribution automatique de points et de mise à jour du niveau utilisateur.

Interface Tkinter

Tkinter n'est pas au programme. Pour la mise en page des vues (sidebar, thème, transitions), nous avons utilisé l'IA comme référence de documentation. La logique métier et les choix UI restent les nôtres.

Flux Général de l'Application



Structure du Projet

ULB-StudyShare/

- ├─ main.py
- ├─ schema.sql
- ├─ requirements.txt
- ├─ .env_example
- ├─ **core/**
 - ├─ auth/
 - ├─ db/
 - ├─ importers/
 - ├─ models/
 - ├─ parsers/
 - ├─ repository/
 - └─ services/

├─ gui/

- ├─ app.py
- ├─ controller.py
- ├─ controllers/
- ├─ views/
- └─ ui_helpers.py
- ├─ **data/**
 - ├─ cours.csv
 - ├─ recompenses.xml
 - ├─ utilisateurs/
 - └─ commentaires.json
- └─ assets/images/

Récapitulatif

Architecture 4 couches

GUI · Services · Repository · MySQL

7 entités, triggers, index

Contraintes d'intégrité complètes

Import multi-format

47 cours · 45 objets · 50 users · 87/94 évaluations

8 requêtes SQL

Toutes implémentées et affichées dans l'application

Interface Tkinter complète

8 vues : login · dashboard · explorer · shop · stats...

MySQL (choix de l'équipe)

Préféré à PostgreSQL — maîtrise + support triggers/CHECK